

NW – 301

6BLOCKTECH

Midterm Lab Activity 5: Step by Step of Creation of ERC20 Token

Full Code

```
// contracts/OceanToken.sol
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.17;

import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/ecd2ca2cd7cac116f7a37d0e474bbb3d7d5e1c4d/contracts/token/ERC20/ERC20.sol";

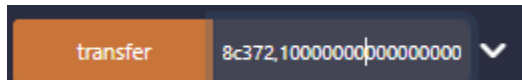
contract Tokenization is ERC20 {

    constructor(uint InitialSupply) ERC20("OceanToken","OCT") { infinite gas 906000 gas
    {
        _mint(msg.sender, InitialSupply);
    }
}
```

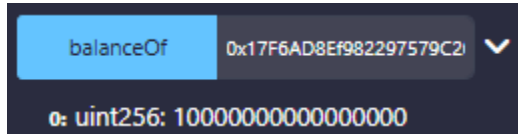
Deploying and Checking balance, name, and symbol.

[illegible]

Transferring 10000000000000000 to a 2nd account

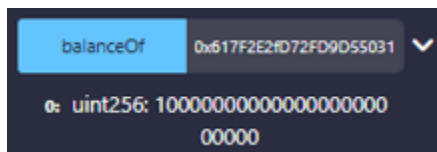


Checking if 10000000000000000 is successfully transferred to the 2nd account

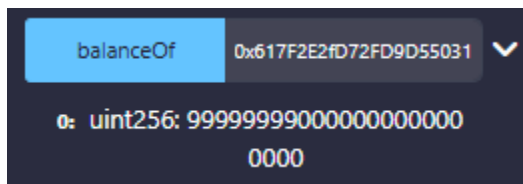


Checking if the first account has reduced balance

Before:



After:



Reflection:

1. **Practical Use:** If you were to launch your own token today, what specific problem would it solve or what "real world" item would it represent?

If I were to create an ERC20 token, it would represent **fitness rewards**. Users would earn tokens by completing healthy activities like workouts, running, or attending wellness classes. The tokens could be redeemed for gym memberships, fitness gear, or wellness services, and could even be traded or gifted to others. This system motivates people to stay active, makes rewards transparent and portable, and leverages blockchain to track and verify healthy habits securely.

2. **Security & Trust:** Why is it safer to use a standard library like **OpenZeppelin** rather than writing your own token code from scratch?

Using **OpenZeppelin** is safer because it's battle-tested, audited, and widely used, reducing bugs and exploits. Writing your own token from scratch risks critical security flaws that attackers actively look for.